



电子科技大学
格拉斯哥海南学院
Glasgow College Hainan, UESTC

UESTC 4014: Real-Time Computing Systems and Architecture

Fall Semester 2026-2027

Lab 1 - Week 1

Report Due Date: Friday, 18th September 2027, 23:59 CST

Student's Name	
Student's UoG ID	
Student's UESTC ID	
Date	

1: Introduction

This lab will give you a more deeper level practical understanding for the STM32 controllers specially for interfacing with external components such as different sensors, actuators and sending and receiving information from other controllers. It contains the following content

- Binary number system and binary operations
- Registers
- Timers and Interrupts

For this lab, each of the above concepts will be explained with code and examples. You will be given tasks through out the lab, for you to implement what you have learned. You will have to create a report explaining how the tasks were accomplished with sufficient figures of what you have done.

2: Binary Number Systems & Operators

The objective of this section is to explore binary operators and binary number systems Understanding binary operators is essential for performing bitwise operations, while familiarity with number systems is crucial for low-level programming in embedded systems.

The binary number system is a fundamental numeral system used in digital electronics and computer programming. It is composed of two digits, 0 and 1, representing different values in positional notation. Unlike the decimal system, which uses base 10, the binary system operates on base 2. All the computers process data with these 0s and 1s.

Each 0 or 1 is considered to be a bit. A binary number of 4 bits can be 0001, 0010, 0011, etc and a binary number of 32 bit can be 100010100100100101001001001000. The STM32 as the name suggests is a 32 bit controller, and its registers are 32 bit long, but more on it in the next section.

As we perform operations on our decimal number systems for example addition, subtraction, binary operators can be used on binary numbers. This section will specifically be focused on bitwise operators as these operators can become very handy while performing register manipulation.

2.1: Bit Wise Operators

The Table-1 shows the list of important bitwise operators and their symbols.

Operator	Symbol
AND	&
OR	
XOR	^
NOT	~
Left Shift	<<
Right Shift	>>

Table 1: Important bitwise Operators

Now let's have a look at how these operators work.

AND Bitwise Operator

This operator works on the AND logic, meaning, the output is one only when both the bits are high or are one i.e. $1 \& 1 = 1$, all the rest combinations will result in 0. Consider the following example:

$$\begin{array}{r} 1001011 \\ \& \\ 1011010 \\ \hline 1001010 \end{array}$$

In the example above, there are two 7bit binary numbers on which AND bitwise operation is performed.

OR Bitwise Operator

The OR operator takes 2 bits and outputs high when one of them is high or one and returns 0, only when both are 0 or LOW, i.e. $1|0 = 1$ and $0|0 = 0$. Let's again consider the above numbers and again perform an OR bitwise operation on them.

$$\begin{array}{r} 1001011 \\ | \\ 1011010 \\ \hline 1011011 \end{array}$$

XOR Bitwise Operator

The XOR operator outputs 1 only when both the bits are different and 0 when both the bits are same. This can be elaborated in the example shown below.

$$\begin{array}{r} 1001011 \\ \wedge \\ 1011010 \\ \hline 0010001 \end{array}$$

The bitwise XOR operator is commonly used for various tasks, including flipping individual bits in a binary number, swapping the values of two variables without using a temporary variable, and checking whether an even number of bits are set to 1 in a binary number.

NOT Bitwise Operator

The NOT operator takes a bit and inverts it, it is also known as the complement operator. Consider the following example:

$$\sim 1001011 = 0110100$$

Left Shift Bitwise Operator

The left shift operator shifts the bits to left by the amount described. Consider the following 7-bit bit binary number.

$$1001011 \ll 3 = 1011000$$

As can be observed, the bits have been shifted by 3 and the left most bits were discarded as the number was 7 bits.

Right Shift Bitwise Operator

The right shift operator is similar to the left shift but shifts bits toward right by the number of bits mentioned as can be observed in the example.

$$1001011 \gg 3 = 0001001$$

Application of bit shift operators in Registers

The bitwise operators come in very handy specially while manipulating registers which are bigger in size like the STM32 32bit registers. Lets take an example of an 8 bit register of which you need to make the 3rd and 5th bit 1 and 0 respectively to preform a specific function. Other bits of the registers are already filled, you do not know their values and you cant change them, so how will you do this. Lets have a look.

Step 1: Creating a binary number mask which have the position 3rd and 5 high.

$$101 \ll 3 = 00101000$$

Step 2: now we invert these by preforming a not operator on this

$$\sim 00101000 = 11010111$$

Step 4: Preforming and bit wise operation with the register to reset the required bits. Here x describes the bits we do not know the value for

$$\text{xxxxxxx} \mid 11010111 = \text{xx0x0xxx}$$

So now you have the required bits reset, without altering the complete register.

Step 5: Again, create a binary mask as in step 1 with the desired values of the bits

$$001 \ll 3 = 00000100$$

Step 6: Preform bitwise OR operator between this binary mask and the register.

$$\text{xx0x0xxx} \mid 00000100 = \text{xx0x1xxx}$$

And hence the register 3rd and 5th bits are reset as required. But its not a good practice to preform the the above operations in 6 cycles, instead we can preform in 2 cycles as shown below with a C code to do so.

Register &= ~(101<<3)

Register |= (001<<3)

This would preform the same function as the above 6 steps.

3: Registers

Registers play a crucial role in the operation of STM32 microcontrollers, acting as essential components for configuring and controlling various aspects of the device. Registers are special memory locations within the microcontroller's Central Processing Unit (CPU) that hold control and status information. They allow direct access to specific hardware features, peripherals, and settings of the microcontroller. Each peripheral or feature often has its own set of dedicated registers that can be read from or written to in order to control its behavior. For the STM32 architecture, the size of registers is 32 bits. STM32 microcontrollers typically feature a rich set of peripherals, including General Purpose Input/Output (GPIO) ports, timers, UARTs, I2C, SPI, ADC, and more. Each of these peripherals is controlled and configured through a series of associated registers. For example, the GPIO ports' data direction, output levels, and interrupt settings are controlled by registers specific to GPIO.

To interact with these registers, developers need to know the addresses and bit patterns of each register at which we will have a look at this section and observe how we can alter some of these registers to perform required tasks. For a detailed view of all the registers of a typical STM32 nucleo board or microcontroller you can refer to this [Reference manual](#).

When programming STM32 microcontrollers, we often use Hardware Abstraction Layer (HAL) library provided by STMicroelectronics as mentioned in lab 0. These libraries offer functions that handle register read and write operations in a more abstract and user-friendly manner, simplifying the coding process, but still sometimes specially for time intensive applications, it might be better to use direct register access as it is much faster than using the HAL library.

3.1: GPIO Registers (GPIOX_CRL & GPIOX_CRH)

Click the above-mentioned Reference manual and go to the section for GPIO registers on page 171. The GPIOX_CRL and GPIOX_CRH are discussed here briefly. It can be seen that both these registers are 32 bit as all of the registers of STM32 are. As mentioned in the manual, these registers are used to initialize the the GPIO ports. Each GPIO port in the STM32 has 16 pins so for example if you want to set a certain pin of the GPIO port A, the register GPIOA_CRL or GPIOA_CRH will be used. The CRL is for the lower 8 pins (0-7) and the CRH are for higher 8 pins (8-15). The GPIOX_CRL and GPIOX_CRH are shown in Figure-1(a) and 1(b).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 1(a): GPIOX_CRL [1]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 1(b): GPIOX_CRH [1]

So, from the information provided above and the manual, it can be inferred that each pin requires 4 bits to be initialized according to the pin and port under consideration. The Table-2 and 3 shows the logic for setting each pin from the GPIO ports.

Configuration Mode		CNF1	CNF0	MODE1	MODE0
General Function Output	Push Pull	0	0	0	1
	Open Drain		1	1	0
Alternate Function Output	Push Pull	1	0	1	1
	Open Drain		1	See Table 3	
Input	Analog	0	0	00	
	Input Floating				
	Input Pull Down	1	1		
	Input Pull Up		0		

Table 2: Port Bit Configuration Table

Mode 1	Mode 0	Meaning
0	0	Reserved
0	1	Maximum Output Speed 10 MHz
1	0	Maximum Output Speed 2 MHz
1	1	Maximum Output Speed 50 MHz

Table 3: Output Mode Bits

In a code, you can set the register as

GPIOX->CRL = 0bXXXXXXXXXXXXXXXXXXXXXXXXXXXX

TASK 1: Based on the information provided so far, your task is to write a small code snippet (for the report only, not for execution in the STM IDE) to configure pin 5 of GPIOB as a general-purpose output with a maximum output speed of 50 MHz. You must show the step-by-step procedure of the bitwise operations, along with the resulting output after each operation. You should also demonstrate how to achieve this in the minimum number of steps, and finally, show how this would be written in your code. [25 Marks]

The above task performs the same function as setting PORTB pin 5 as an output using either the GUI or the HAL library.

3.2: GPIO Registers (GPIOX_ODR)

In the reference model on the page number 173, you will be able to see GPIOX_ODR which is also shown in Figure-4.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Figure 4: GPIOX_ODR [1]

The GPIOX_ODR is used to change i.e. to set or reset any specific pin of a GPIO port and can be used to write bits on the pins when the pin is in output mode. As can be inferred from the Figure-2, the bits 16 to 31, are set to reserved, i.e. we can't change them, and they are being used by the system. The bits 0 to 15 are read and write bits and corresponds to the port pins, so for example if we set GPIOA_ODR bit 3 to 1 or high, the output on that pin will be high.

The advantage of using the register programming instead of the HAL library is that it is much faster and can be advantageous with time intensive applications.

Now let's see the *blink* example from the lab0 but instead of using the HAL library to set and reset the led pin, let's use the GPIOX_ODR register. As done in the blinky example set the pin PA5 as output in the graphical layout of the controller and in the while (1) let's try to play with GPIOA_ODR register.

Now as we want to toggle the 5th pin of PORTA, we reset the value of GPIOA_ODR as

```
GPIOA->ODR &= ~(0b1<<5)
```

Now to set the the bit high and in return setting the pin high

```
GPIOA->ODR |= (0b1<<5)
```

And to set the bit low

```
GPIOA->ODR |= (0b0<<5)
```

3.2: GPIO Registers (GPIOX_IDR)

The GPIO register IDR is very similar to the ODR, but it is used to read the values on the bits instead of writing on it and hence can be used when the pins are in input mode. The GPIOX_ODR is shown in the Figure-5.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Figure 5: GPIOX_IDR [1]

As can be seen in the Figure-5, similar to GPIOX_ODR, the IDR bits 16 to 31 are reserved and the pins 0 to 15 correspond to the pins to the ports which have to be read. Consider you are to read the input from the from the blue button and save it in variable. You can do it as

```
bool Pinx = (GPIOx->IDR & (1 << GPIOx_PIN_N)) != 0
```

So the pinx will save true state of the pin and false is the pin state in 0.

Note: “bool” in the above code may give you the following error:

```
../Core/Src/main.c:81:3: error: unknown type name 'bool'
```

This is because the newest IDE does not define bool type

To overcome this issue, please add the following code in the #include part

```
#include <stdbool.h>
```

TASK 2: Explain how the above command works, including steps and an example. [20 Marks]

TASK 3: Write code using the *GPIOX_IDR* and *ODR* registers to turn the built-in *LED* on when the blue button is pressed and off when it is released. [20 Marks]

There is a plethora for STM32 and most of them can be accessed directly by the help of the HAL library and direct register manipulation as described above. Both methods are fine according to the needs of the developer. By now you should have an idea how the registers are manipulated to make the microcontroller preform a certain task. You can have a look at the reference manual to have a look at all the registers and understand how they work.

4: Timers & Interrupts

4.1 Timers Introduction

Timers are essential peripherals of any microcontrollers and can provide accurate timekeeping and timing functionalities. They are highly versatile and can be configured for a wide range of tasks, including generating precise time intervals, measuring the duration of events, generating PWM signals, and more. Timers are crucial for many applications, such as controlling motors, capturing sensor data, generating communication signals, and managing real-time tasks. To understand the basic functionality of a timer, consider the Figure-6.

The Figure-6 shows the three main components for timers. The first is the main CPU clock, which provides pulses of a certain frequency, in the figure above say it is X MHz. The prescaler divides the clock provided by the

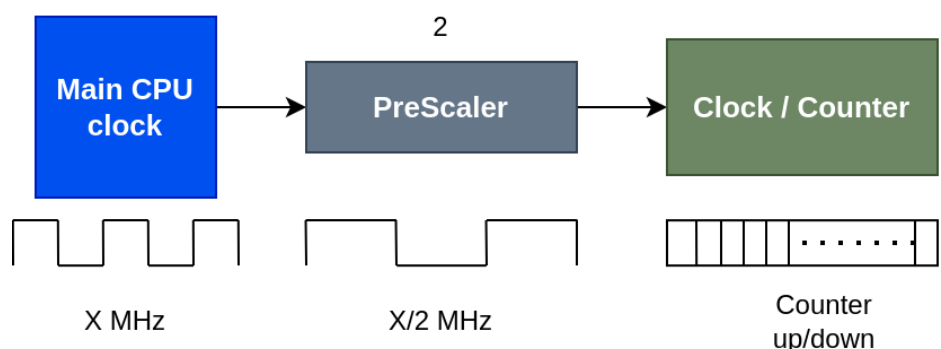
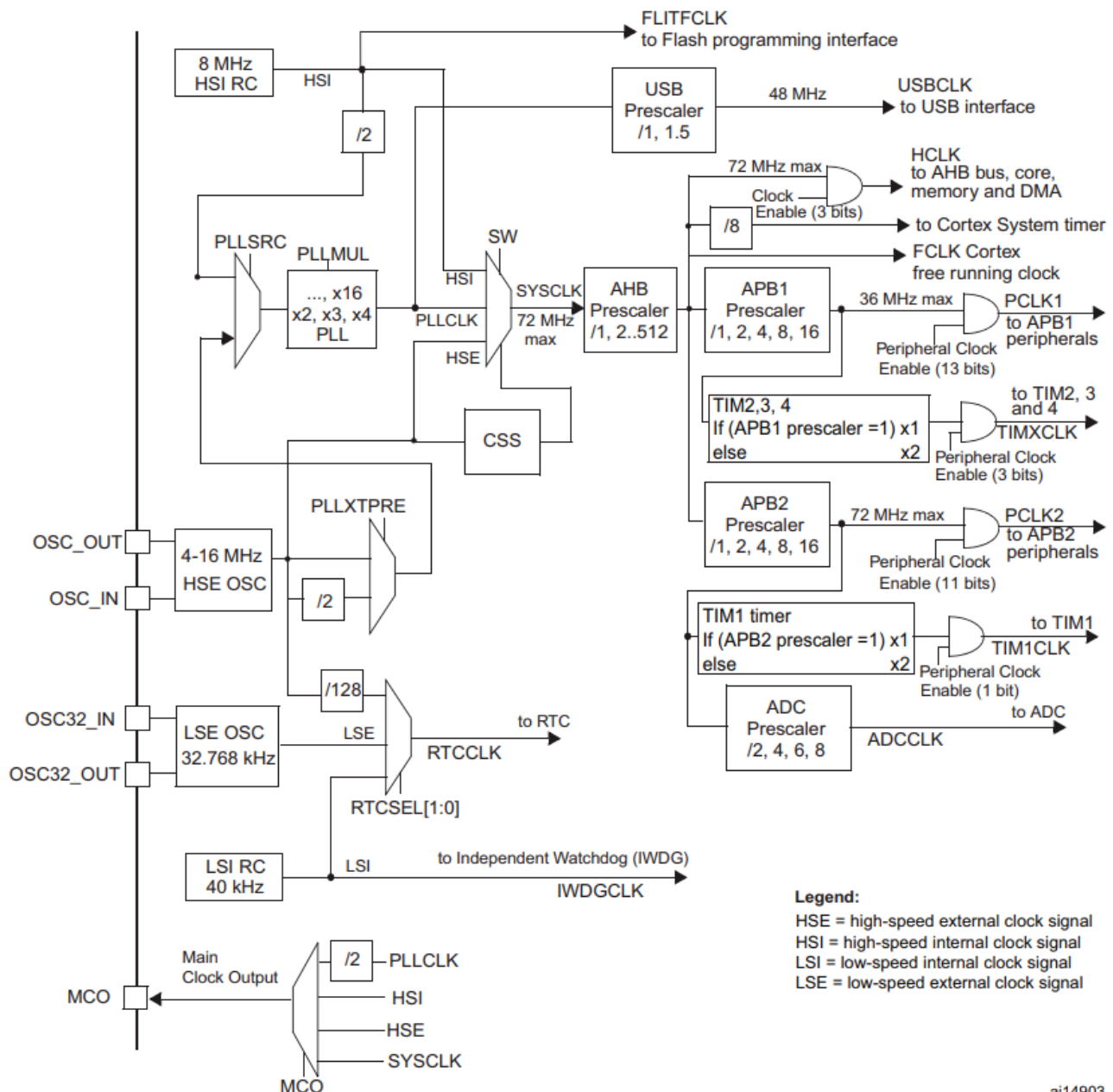


Figure 6: Basic components of a Timer

CPU by a certain value to decrease the frequency according to the needs of the developer. Upon every pulse from the prescaler, the value of the counter can be increased or decreased. When the counter reaches a specific value, (adjusted by the developer) different events can be driven and the counter resets and this is how you can get accurate time driven events.

The Figure-7 shows an elaborate clock tree of for the STM32 F103RB. It can be observed from the clock tree, that there are multiple timers with different prescalers and are connected to the CPU clock. For now we will have a look at the TIM2, 3 and 4 which are 16 bit, each with up to 4 IC/OC/PWM or pulse counters and incremental encoder input.



ai14903

Figure 7: CLOCK Tree STM32 f103rb [2]

Example: If you have a clock frequency of 80 MHz and want an event to be driven after 2.5 seconds what should be the values for the prescaler and the counter such that both the counter and the prescaler are 16 bits.

Solution: Considering that the counter and prescaler are 16 bits the maximum values that it can reach are $2^{16} - 1 = 65,535$ so both the values should be between this.

Now if the value of the prescaler is selected to be 80 the frequency after the prescaler is

$$\text{prescaler frequency} = \frac{80 \text{ MHz}}{8000} = 10 \text{ KHz} = 10000 \text{ Hz}$$

This means that the prescaler would give out a pulse after every 0.0001s. Now to set the value of the counter we just need to know how many pulses will it take to reach 5.5 sec hence

$$\text{Counter Value} = \frac{2.5}{0.0001} = 25,000$$

Hence for a CPU clock of 80 MHz, if the prescaler value is 16 and counter value is 25,000, we can drive an event after 2.5 seconds.

4.2 Getting Started with Timers

Now let's get started with actually using the timers in the STM32 nucleo board. Make a new project and in the graphical interface, click the clock configuration. You will be able to see the clock configuration for the microcontroller. This is shown in Figure-8.

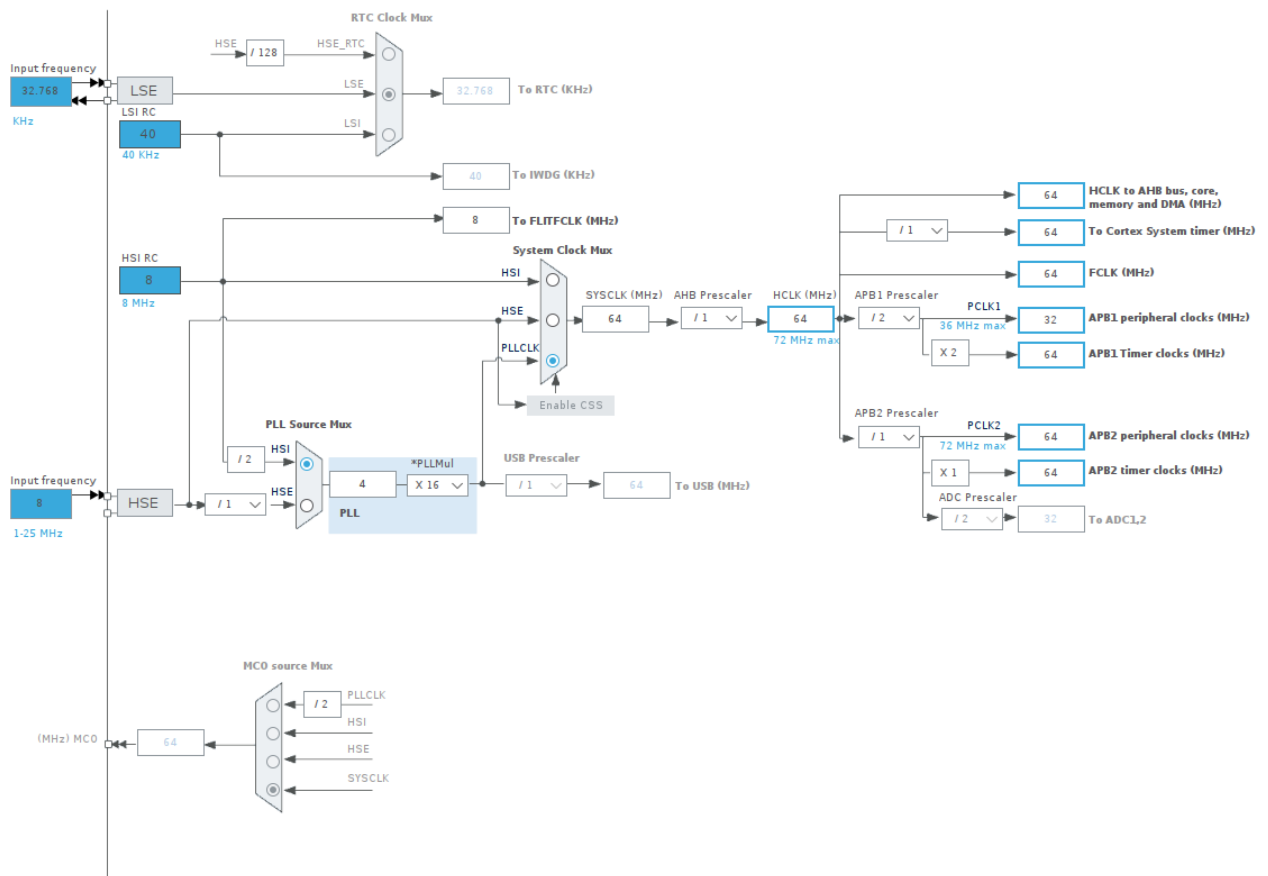


Figure 8: Default Clock Configuration

As can be seen from the image, the AHB prescaler is 1 and the default clock pulses are set to 64 MHz going to a maximum of 72 MHz. You can adjust this using the AHB prescaler, but for now we will let everything to be as default.

Go back to the pinout & configuration and select the timer drop-down menu as shown in Figure-9. Here you can see there are different timers i.e. RTC, TIM1, TIM2, TIM3 and TIM4. These are the names of all the timers. Select the TIM2 and select the Clock Source as internal clock, which means we will be using the internal clock of STM32 for the functionality of TIM2. If you click any channel, you will be able to different

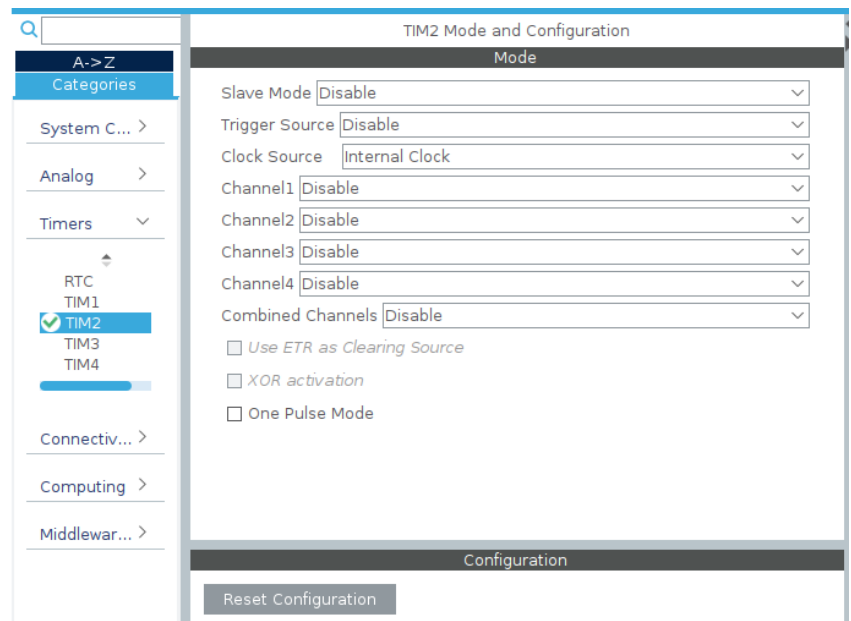


Figure 9: Timer Mode Selection

functionalists that can be performed. These are

- Input Capture Direct Mode: Stores the value of the timer counter when an event happens on a pin.
- Output Compare lets us toggle a pin whenever the timer counter reaches a set value.
- PWM: Lets us produce a pulse Width on a pin.

For now, let everything to be default except for the clock course.

Now, we can look at the configuration of the TIM2, which are shown in Figure-10.

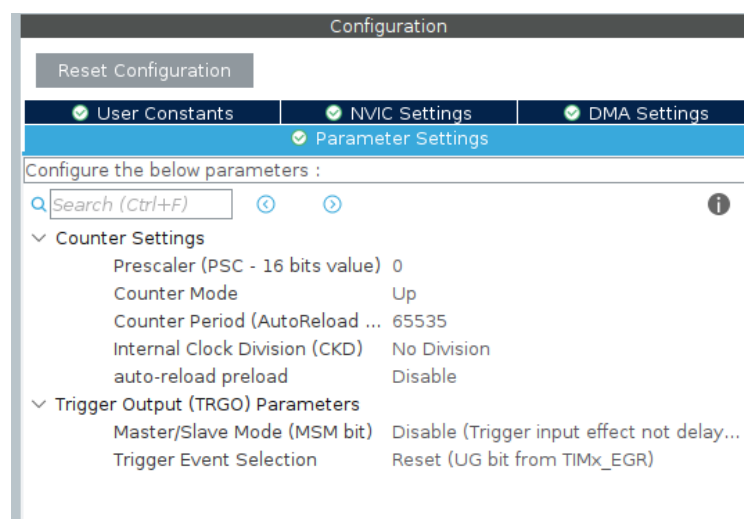


Figure 10: Timer Mode Selection

In the counter settings we you can see the following settings

- Prescaler: Used to divide the input clock with a certain value can is 16 bits.

- Counter mode: To count the counter in increments or decrements with every pulse.
- Counter Period: The maximum the counter can increment up to and then reset.
- Incremental Clock Division: If the clock is to be divided even more.

We can try to make the blinky code again, but instead of the delay function, use timers. Initially we need to calculate the values of prescaler and counter period to produce the timer to toggle the led after every one second.

So, we know the clock is 64 MHz, as the prescaler 16bit, it can have a value of up to 65535. **One thing to be noted is that the prescaler that is entered in the configuration settings is always one less than the actual prescaler.** We select the prescaler to be 65 and put 64 in the configuration. Now we must calculate the value of the counter to produce a timer of 1 sec, so:

$$\text{prescaler frequency} = \frac{65 \text{ MHz}}{65000} = 1 \text{ kHz} = 1000 \text{ Hz}$$

So, the counter value

$$\text{Counter Value} = \frac{1}{0.001} = 1000$$

Hence in the configuration settings, we put

Prescaler = 65000 - 1
Counter mode = up
Counter Period = 1000 - 1

To get a delay of 1 second and let the rest remain default. Make sure that the PA5 (The internal LED pin) is set as output and save the file to generate the code. The HAL library will automatically generate the code for the timer we set.

As can be seen, the IDE

automatically put the TIM2_Init() function in the code, and if you go in the definition of this function you will be able to see how the it initializes the timer and the values we initialized in the configuration settings . To start the timer, we use the HAL_TIM_Base_Start(&htim2). The Figure-12 shows the snippet for getting the counter values and how they can be used to set the toggle the led pin after everyone second.

```

88
89  /* USER CODE END SysInit */
90
91  /* Initialize all configured peripherals */
92  MX_GPIO_Init();
93  MX_USART2_UART_Init();
94  MX_TIM2_Init();
95  /* USER CODE BEGIN 2 */
96

```

Figure 11: TIM2 Init function

```

// starting the timer
HAL_TIM_Base_Start(&htim2);

// setting the timer value
timer_val = __HAL_TIM_GET_COUNTER(&htim2);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    // Checking the value of the counter if reached the counter period
    if ((__HAL_TIM_GET_COUNTER(&htim2) - timer_val) >= 1000)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
        timer_val = __HAL_TIM_GET_COUNTER(&htim2);
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

```

Figure 12: TIM2 Init function

Initially the value in the counter is taken after it is started by the help of `__HAL_TIM_GET_COUNTER(&htim2)` and saved into a `uint_16t` variable `timer_val`. After this in the `while(1)` and is compared with the current value of the counter. If they are more than 1000 which is the counter period, it toggles the led and then saves the value in the timer value and this goes on.

Unlike using the `delay()` function which is a blocking element, i.e., in the system does not do anything while the delay is executed, using the timers is a nonblocking method, meaning, that while we wait for the led to execute, we can execute other lines as well, and when the counter reaches its values, it will toggle the led.

Note: The above mentioned code is not the right way to use timers to execute a time driven events, the reason being, it is possible that sometimes, when the control reaches the if condition, the counter is not at the desired value and it misses the loop and when it again reaches the if condition, it already reset and hence the event is never executed. Now that you have understood the timers, we can move to the right way to doing events, i.e. using interrupts.

4.3 Interrupts

Interrupts are mechanisms that enable a microcontroller to halt its normal execution temporarily and handle a specific event. These events can range from external signals (e.g., button presses, sensor outputs) to internal triggers (e.g., timer overflows, data transfer completions). When an interrupt event occurs, the microcontroller temporarily suspends its current task, saves its context, and switches to executing a predefined interrupt service routine (ISR) to handle the event. After the ISR is executed, the microcontroller resumes its original task.

4.3.4 Timer Interrupts (Internal)

Timer interrupts are the interrupts that are called when a certain timer counter reaches the Counter Period value and hence executes the commands at exact times and as in the problem discussed above, it does not have to go in the if statement, rather it takes the control to the ISR where the commands are to be executed making it timelier active.

Now, we can do the blinky program correctly using the internal interrupt and timers. Let's first see what happens every time the counter reaches the counter period and resets. It is elaborated in the Figure-14.

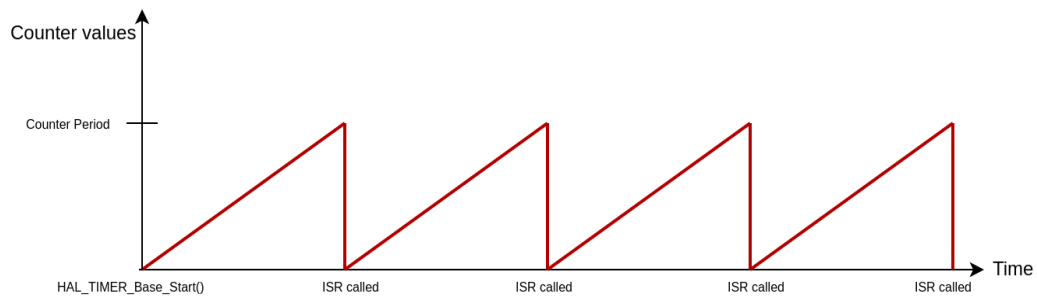


Figure 14: internal timer Interrupt working

As it can be observed from the Figure-14, when the timer starts, the values of the timer counter keep on increasing, once the counter reaches the counter period, the counter is reset to 0 and the control transfers to the ISR which can have any time dependent code, like for our example, it will be the blinky code. After the ISR is completed, the control will go back to the where the control was before the interrupt until the interrupt is called again.

To setup the timer, go back to the pinout and configuration, and from the timer dropdown select the tim2. In the parameter settings let the values as they were in the previous section, as they are already set for one second. Click the NVIC settings and enable the TIM2 global interrupt, as shown in the Figure-15, without this the tim2 interrupt will not work. Save the file, so that the STM IDE can generate the code according to the new settings. Now delete everything in the main.c that was in the main loop as now everything will be in the ISR and we won't need any code in the main loop.

One thing to remember is that as we are now using the timer in the interrupt mode, instead of `HAL_TIM_Base_Start(&htim2)`, we will be using `HAL_TIM_Base_Start_IT(&htim2)` to start the time with interrupt.

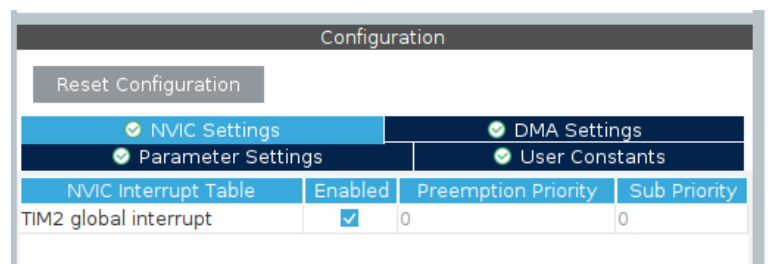


Figure 15: Setting up the timer interrupt

Navigate down to where it says USER CODE BEGIN 4, this is where we will write the call back function for the interrupt that is called when the counter resets. The code snippet for the call back function is shown in figure 12. One thing to be noted is that the HAL API handles the ISR. Now the call

```
272 /* USER CODE BEGIN 4 */
273 // call back for the timer 2 reset
274 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
275 {
276     if(htim == &htim2)
277     {
278         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
279     }
280 }
281 /* USER CODE END 4 */
282
```

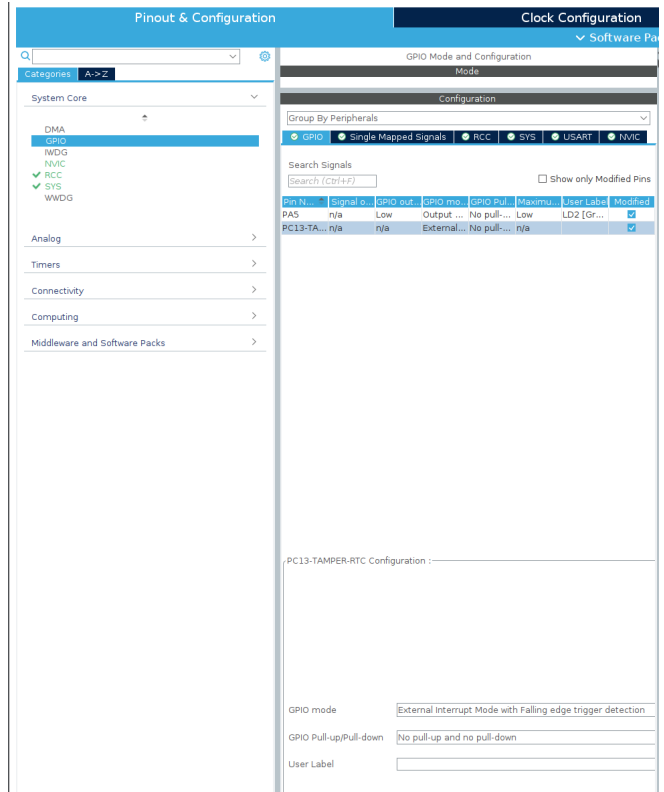
Figure 16: Interrupt Callback Function code snippet

back function that is used in the code for the interrupt is `HAL_TIM_PeriodElapsedCallback()`, which is a fairly generic function hence it is a good idea to describe due to which interrupt was it called, which is described as `htim == &htim2`, because timer 2 interrupt is being utilized here, if that checks out we just toggle the GPIOA PIN 5 and hence we have the blinky code using the timers and interrupt.

4.3.4 Interrupts (External)

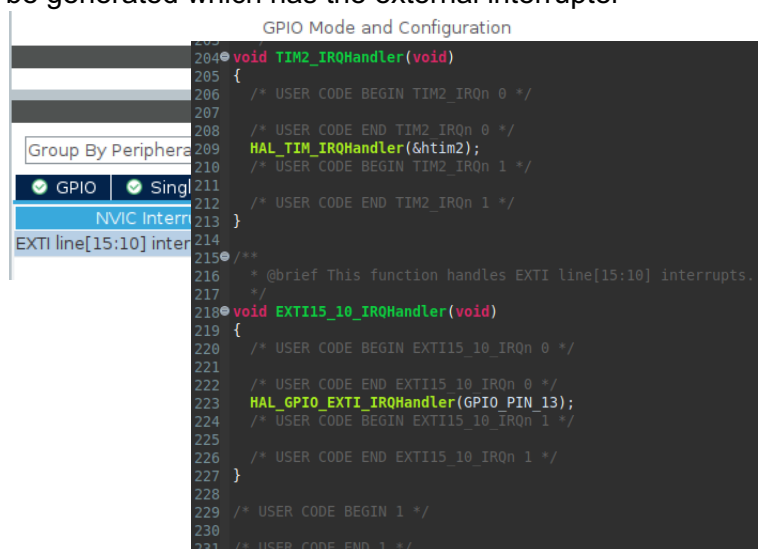
Like the internal interrupts you worked with above, there are external interrupts as well that depend on the GPIO pins of STM32. For example whenever a falling edge, rising edge or a changing edge is detected in on a specific GPIO, the ISR will be called instantly and the required code will be executed. Lets try to toggle the LED using the external GPIO interrupts such that when ever the button is pressed, the LED toggles.

In the pinout & configuration settings, click PC13 and from the dropdown menu select the `GPIO_EXTI13`, which will set this pin as an interrupt pin. Now lets select on what pin state do we want the ISR to be called. To do so, clock the GPIO under the system core. This will show all the active GPIOs. Here click PC13, and you will be able to see the configurations for this pin. In the GPIO mode, from the drop down menu, you can select all the modes on which you will be able to get the interrupt. Lets select “External Interrupt mode, with falling edge trigger detection”. This means, that the interrupt will only be activated, when the pin detects a falling edge. This complete setting is shown in figure 13.



Next select the NVIC and enable the `EXTI_line[15:10]` interrupts. This enables the interrupt like we did for the timer2 interrupt above. This is shown in figure 14

Now save the file and a new code will be generated which has the external interrupter connected to PC13 activated.



Now if you want to have a look at all the interrupts you have activated you can navigate, in the project explorer to `core>source>stm32f1xx_it.c`. This file consists of all the interrupts that have been activated. If you scroll down you will be able to see the timer and the external interrupt handlers. This is shown in figure 15. Now for example, you can see in the external interrupt handler, there is another function for `GPIO_PIN_13`, which is out selected interrupt pin. Now if you right click this function, and click “open Declaration” the IDE will take you to the exact working of this function. This is shown in figure 16.

As can be seen, the handler, calls another function, i.e. the `HAL_GPIO_EXTI_Callback()`, which is the actual function where we can write the code, which we want to execute when the pin changes the desired state. So we copy this function to our `main.c` and write the desired lines as was done for the timer interrupt. In the main function we write this function and in the function write the code

Figure 15 : Interrupt file

to toggle the led every time, the the interrupt is called. This is shown in figure 17.

Now you know the basic functionality of timers and interrupts. To dig in deeper, you can have a look at the [STM32 timer cookbook](#) and [HAL_Lowlayerdrivers](#).

```
/**
 * @brief This function handles EXTI interrupt request.
 * @param GPIO_Pin: Specifies the pins connected EXTI line
 * @retval None
 */
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    /* EXTI line interrupt detected */
    if ( __HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
    {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}

/**
 * @brief EXTI line detection callbacks.
 */
//void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef)
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}

/* USER CODE END 4 */
```

TASK 4: Write code using timers and external/internal interrupts to blink the built-in LED for 1.5 seconds when the button is pressed once, and to turn it off when the button is pressed again. You are not allowed to use the `delay()` function. You should provide the following in your code:

- The prescaler
- The timer
- The counter period
- The code

[35 Marks]

Hint : `HAL_TIM_Base_Stop_IT` Can be used to stop the timer and the interrupts.

References

- [1] https://www.st.com/resource/en/reference_manual/rm0008-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] https://www.st.com/resource/en/application_note/dm00236305-generalpurpose-timer-cookbook-for-stm32-microcontrollers-stmicroelectronics.pdf
- [3] https://www.st.com/resource/en/user_manual/um1850-description-of-stm32f1-hal-and-lowlayer-drivers-stmicroelectronics.pdf